

Build Phase 3 — Proctor Guide

Your Role

Phase 3 is where Playwright becomes deeper learning instead of first exposure. Your job is to **help** people choose achievable higher-value test scenarios, keep them from over-scoping, and **start nudging** people toward demo prep in the last 15 minutes.

Timeline Check-ins

Time Into Phase	Where They Should Be	Red F
-----	-----	-----
10 min	Picked their next scenario; negative-path test from Phase 2 either passing or actively being fixed	Still re
30 min	Negative-path test passing; deeper scenario started	Only h
1 hour	Deeper scenario or debugging exercise underway	Still on
1.25 hours	Scenarios wrapping up; starting demo prep and documentation step	Scena
1.5 hours	App running clean; demo rehearsed; `README.md`, `TESTING.md`, and `techOverview.md` drafted	Nothing

Helping People Choose

If someone asks "What should I do?", guide them based on where they are:

Situation	Recommended Next Step
-----	-----
Only happy-path test passes	Add a negative-path validation test first
Wants a realistic demo	Add backend failure handling or a richer business-rule scenario
Comfortable with file I/O	File upload test — high impact but higher risk
Still shaky on Playwright	Do the debugging exercise with traces/screenshots instead of adding app scope
Finished early	Optional stretch goal outside Playwright

****Steer people away from big app changes**** unless the testing value is obvious. The goal is deeper Playwright learning, not feature sprawl.

Common Issues

Negative-Path Test Carried Over Incomplete from Phase 2

Some teams will enter Phase 3 with a negative-path test that's only partially working. This is normal.

****Nudge:**** "Before adding anything new, finish the negative-path test from Phase 2. One clean failing-case assertion is far more demo-able than two half-built tests."

Library Installation Failures

For deeper scenarios like file upload, participants may struggle with:

- ``pdf-parse`` / ``pdfplumber`` installation issues
- PDFBox Maven dependency resolution
- Binary dependencies for Tesseract (if they attempt OCR — discourage this)

****Nudge:**** "If the library isn't installing in 5 minutes, pivot to a debugging scenario or another test case. Don't spend the phase debugging package installs."

Over-Scoping

Watch for people trying to do 3+ new scenarios or major app rewrites. One strong additional test is better than three half-finished ideas.

****Nudge:**** "Pick one and finish it — a stable negative-path or deeper test demos much better than three broken scenarios."

Forgetting Demo Prep

With 15 minutes left, actively remind people:

"Start wrapping up! Think about what you want to show in your demo. You have 5 minutes to present — what's the story?"

Missing Final Documentation

Some teams finish tests but forget to capture run/test instructions and architecture notes.

****Nudge:**** "Before the demo, create ``README.md``, ``TESTING.md``, and ``techOverview.md`` so someone else can run this project without guesswork."

Weak Assertions

Some teams will add more tests but keep weak assertions like "page contains text."

****Nudge:**** "Can you assert the exact validation message, status banner, or field-level error instead of a vague text check?"

Last 15 Minutes: Demo Prep

Phase 3 Step 6 is a dedicated demo-prep step for participants. Your job is to enforce it.

Actively walk around (or message in chat) and say:

1. ****"Stop coding now."**** Get the app in a working state.
2. ****"What are you showing?"**** Which screens, which test output, which Copilot moment.
3. ****"Is your test data ready?"**** They should know exactly what inputs they'll enter live.
4. ****"Restart the app and run your tests once more"**** — verify it works clean from scratch.
5. ****"Capture your docs now."**** Ensure ``README.md``, ``TESTING.md``, and ``techOverview.md`` each have usable content.

6. **What's your 5-minute story?** App → tests → best Copilot moment → what surprised you.

Phase 3 Success Criteria

It's OK if: They completed one strong additional Playwright scenario, the negative-path test is passing, they have a clear 5-minute demo story, and the three documentation files are usable — even if deeper scenarios were only partially explored.

It's not OK if: The app is broken for the demo, they never progressed past the happy-path test, demo prep was skipped entirely, or there is no usable run/test documentation.

Facilitator One-Page: Variation Matrix (All Phases)

Use this during check-ins to recommend 2-3 experiments per phase without slowing teams down.

Phase	Pick 2-3 Variations	Fast Proctor Cue
---	---	---
Phase 1: working flow first	Test-First Variation, Selector Quality Challenge, Prompt Detail A/B Test	"Get one stable pass"
Phase 2: correctness and confidence	Failure Injection Drill, Explain-Back Check, Role-Based Prompting	"Now stress the app"
Phase 3: polish and demo strength	Model Comparison, Refactor Pass, Demo Readiness Variant	"Choose the cleanest"

30-Second Recommendation Script

If a participant asks what to do next, use this:

- "Pick one reliability variation and one communication variation."
- "Reliability means selector quality or failure injection."
- "Communication means explain-back or demo-readiness script."
- "If you still have time, do model comparison on one prompt."

Decision Rules by Situation

- If they are unstable or flaky: prioritize **Selector Quality Challenge** and **Failure Injection Drill**.
- If they are slow due to rework: prioritize **Prompt Detail A/B Test**.
- If they are close to demo time: prioritize **Refactor Pass** and **Demo Readiness Variant**.
- If they finish early: run **Model Comparison** on one completed checklist prompt and keep the better result.